

Security Scan Report

Target: /fake/target

Scan Date: July 23, 2026 at 14:13 UTC

Scan ID: 4201b66c-cb9f-468b-bd19-cf2417093c22

0 CRITICAL

1 HIGH

0 MEDIUM

2 LOW

Executive Summary

Security scan of '/fake/target' identified 4 raw findings, reduced to 3 unique issues after deduplication. ■ 1 HIGH-severity issue(s) should be prioritized for remediation within the next 48 hours. 2 LOW-priority issue(s) have been logged for backlog tracking.

Scan Statistics

Metric	Value
Total Raw Findings	4
After Deduplication	3
Critical Findings	0
High Findings	1
Medium Findings	0
Low Findings	2
Scan Duration	2.1s

Detailed Findings

[HIGH] #1: Sql Injection in users.py:42

Risk Score	0.5488
Affected Location	app/routes/users.py:42
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89
Detected By	semgrep

Explanation
SQL Injection occurs when user-controlled input is directly concatenated into SQL queries, allowing an attacker to manipulate the query logic. This can lead to unauthorized data access, modification, or deletion of database records.

Code or HTTP Excerpt

```
42 | query = f"SELECT * FROM users WHERE id = {user_id}"
```

Business Impact
An attacker could read, modify, or delete all data in the database, potentially exposing customer records and sensitive business data.

Fix Recommendation
Rule-based recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.

LLM recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.

[LOW] #2: Xss in render.py:15

Risk Score	0.3478
Affected Location	app/utils/render.py:15
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Business Impact
An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[LOW] #3: Hardcoded Secret in settings.py:8

Risk Score	0.3168
Affected Location	config/settings.py:8
Vulnerability Type	HARDCODED_SECRET
Detected By	semgrep

Explanation
Hardcoded secrets (passwords, API keys, tokens) in source code are a critical security risk. Anyone with access to the codebase can extract and misuse these credentials.

Business Impact
Exposed credentials could allow unauthorized access to external services, databases, or cloud infrastructure.

Fix Recommendation
Rule-based recommendation: Move all secrets to environment variables. Use python-decouple or python-dotenv to load from .env files. Add .env to .gitignore. Rotate any credentials that were previously committed.

LLM recommendation: Move all secrets to environment variables. Use python-decouple or python-dotenv to load from .env files. Add .env to .gitignore. Rotate any credentials that were previously committed.

Appendix

Deduplication Statistics

Total raw findings: 4 | After deduplication: 3 | Reduction: 25%

Scan Metadata

Scan ID	4201b66c-cb9f-468b-bd19-cf2417093c22
Target	/fake/target
Timestamp	2026-07-23T14:13:48.173755+00:00
Duration	2.1s
Scanners Used	semgrep
Scanner Versions	semgrep=1.159.0

Raw Findings Data (JSON)

```
[
  {
    "id": "b79e9d70-fe2e-4f69-abaf-c281c10e75db",
    "title": "Sql Injection in users.py:42",
    "file": "app/routes/users.py",
    "line_start": 42,
    "line_end": null,
    "severity": "HIGH",
    "vuln_type": "SQL_INJECTION",
    "description": "SQL injection via string concatenation",
    "rule_id": "python.lang.security.sqli",
    "confidence": 0.9,
    "cwe_id": "CWE-89",
    "cvss_score": null,
    "code_snippet": "query = f\"SELECT * FROM users WHERE id = {user_id}\"",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.5488,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SQL_INJECTION",
    "llm_explanation": "SQL Injection occurs when user-controlled input is directly concatenated into SQL queries",
    "fix_suggestion": "Use parameterized queries instead of string concatenation. In Python, use placeholders like %s.",
    "business_impact": "An attacker could read, modify, or delete all data in the database, potentially exposing sensitive information.",
    "raw": {}
  },
  {
    "id": "d69849fa-88d3-4687-be20-945b5e18866f",
    "title": "Xss in render.py:15",
    "file": "app/utils/render.py",
    "line_start": 15,
    "line_end": null,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Unescaped user input in template",
    "rule_id": "python.flask.security.xss",
    "confidence": 0.7,
    "cwe_id": "CWE-79",
    "raw": {}
  }
]
```

```

    "cvss_score": null,
    "code_snippet": null,
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3478,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other unauthorized actions",
    "raw": {}
},
{
    "id": "cf192b2e-6389-4ba3-a5dc-c7332b36aa42",
    "title": "Hardcoded Secret in settings.py:8",
    "file": "config/settings.py",
    "line_start": 8,
    "line_end": null,
    "severity": "LOW",
    "vuln_type": "HARDCODED_SECRET",
    "description": "Hardcoded password detected",
    "rule_id": "python.lang.security.hardcoded-password",
    "confidence": 0.4,
    "cwe_id": null,
    "cvss_score": null,
    "code_snippet": null,
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3168,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Hardcoded secrets (passwords, API keys, tokens) in source code are a critical security vulnerability",
    "fix_suggestion": "Move all secrets to environment variables. Use python-decouple or python-dotenv to load secrets from .env files",
    "business_impact": "Exposed credentials could allow unauthorized access to external services, databases, or internal systems",
    "raw": {}
}
]

```